Laboratório 3 de CSI-28
Relatório Descritivo - Estoque CASD

Grupo: Arthur de Sousa Vianna, Danilo Carlos da Silva, João Victor Canedo Paranhos, Matheus Lima Reis
Turma: COMP28 **Data:** 23/09/2026

---

1 O Sistema

1.1 Descrição

A aplicação em desenvolvimento é um sistema *web* para gestão do estoque do Centro Acadêmico Santos Dumont (CASD). O sistema tem como propósito digitalizar o controle de itens dessa iniciativa acadêmica, permitindo que seus membros gerenciem o cadastro e a quantidade dos itens estocados, enquanto usuários externos à zeladoria do centro acadêmico podem visualizar o estoque disponível para empréstimo e sinalizar sua utilização.

Atualmente, o controle de itens é realizado de forma manual, em papel, o que dificulta acompanhar a disponibilidade de cada item e identificar quem está o usando, resultando em perda de itens e utilização prolongada.

O estoque é dividido em dois conjuntos: o estoque geral do CASD, formado por itens que podem ser emprestados a alunos, e o estoque interno da zeladoria, de itens que não podem ser emprestados. A aplicação também propõe o gerenciamento e rastreamento de itens internos da zeladoria.

Em relação às tecnologias propostas para a arquitetura do sistema, o *frontend* e o *backend* serão desenvolvidos em Python com o *framework* Django. Para persistência dos dados, será utilizado um banco de dados PostgreSQL hospedado em um serviço de nuvem (Supabase ou Neon). Já a aplicação *web* será hospedada no serviço de *deployment* Render.

1.1.1 Abrangência

O sistema contemplará as seguintes funcionalidades principais, detalhadas no Documento de Requisitos do projeto:

- Autenticação de administradores e diferenciação de permissões entre perfis de usuário;
- Cadastro, edição e remoção de itens do estoque (restrito a administradores);
- Visualização do estoque interno da zeladoria (restrito a administradores);
- Sinalização da utilização de itens do estoque interno da zeladoria, identificada pelo cadastro de administrador, providenciando tempo previsto de utilização (restrito a administradores);
- Sinalização da devolução de itens utilizados (restrito a administradores);
- Visualização de listagem de empréstimos ativos e atrasados (restrito a administradores);
- Visualização do estoque geral do CASD e da disponibilidade de cada item para empréstimo;
- Sinalização, por usuários comuns, do empréstimo de itens, fornecendo identificação mínima (nome, apartamento e e-mail) e tempo previsto de utilização;
- Sinalização, por usuários comuns, da devolução de itens de empréstimo;
- Confirmação de devoluções sinalizadas por usuários comuns (restrito a administradores);

- Geração de um *QR code* para cada item do estoque geral do CASD, direcionando à página de sinalização de utilização ou devolução do item correspondente (restrito a administradores);
- Alertas, via e-mail, para usuários comuns em caso de utilização de um item além do período previsto.

As seguintes funcionalidades são candidatas a expansões futuras, mas está fora do escopo inicial deste projeto:

- Visualização de itens à venda do estoque interno da zeladoria;
- Compra, por usuários comuns, de itens à venda;
- Visualização de informações de vendas dos itens do estoque interno da zeladoria, como quantidade vendida por mês, classes de itens vendidos e receita (restrito a administradores).

O sistema será independente e totalmente autocontido, sem integração com outros sistemas do CASD ou do ITA em sua versão inicial.

1.1.2 Descrição dos Usuários

O sistema atenderá dois perfis de usuário, com níveis de acesso distintos:

- Administradores: membros e funcionários do CASD responsáveis pela gestão do estoque. Membros desejam fácil e rápida visualização do estoque geral do CASD e do estoque interno da zeladoria para auxiliar na organização da iniciativa. Funcionários utilizam itens do estoque interno com frequência. Membros necessitam do controle dos empréstimos do estoque geral e da utilização de itens do estoque interno para evitar a perda e a utilização prolongada dos itens. Acessam o sistema mediante autenticação.
- Usuários comuns: alunos externos à zeladoria do CASD. Utilizam itens emprestados do estoque geral do CASD com frequência. Desejam saber de forma fácil e rápida se o item requisitado está disponível para empréstimo no momento. Acessam o sistema sem autenticação, identificando-se por nome e apartamento ao sinalizar o uso de um item.

1.2 Modelo de Processo

Adotou-se para o desenvolvimento do sistema o modelo incremental. Nesse modelo, a aplicação é desenvolvida em incrementos, de modo que o primeiro concentra as funções essenciais do produto e os subsequentes agregam novas funcionalidades a ele. Cada incremento é entregue de forma finalizada e posto em operação após a conclusão do seu desenvolvimento, ajudando a conhecer os requisitos dos incrementos posteriores.

A escolha se deve, principalmente, à possibilidade de os administradores ganharem experiência com o sistema antecipadamente, fornecendo *feedback* que orienta os incrementos seguintes. Outra motivação para a escolha desse modelo foi a redução do risco de falha geral do projeto, visto que *software* funcional é entregue a cada incremento, não apenas ao final do desenvolvimento.

Foram planejados os seguintes incrementos, dos quais o primeiro constitui uma versão base de uso interno do CASD, ainda sem autenticação:

1. Produto essencial: implementação da interface do usuário e do banco de dados do estoque, integração do *site* com o banco de dados e visualização dos estoques e cadastro, edição e remoção de itens;
2. Permissões: autenticação de administradores e diferenciação de permissões entre perfis de usuário;

3. Registro de empréstimos: sinalização de utilização, de empréstimo e de devolução, geração de *QR codes*, controle em *dashboard* da disponibilidade dos itens e da listagem de empréstimos ativos;

4. Acompanhamento de devoluções: envio de alertas por e-mail em caso de utilização de um item além do período previsto, listagem de empréstimos atrasados e confirmação de devoluções por administradores.

1.3 Arquitetura

A Figura 1 exibe a arquitetura do sistema proposta elaborada com base nos requisitos identificados. A aplicação *web* consiste de uma arquitetura monolítica sob o padrão MVC (*Model-View-Controller*), escolhida devido à simplicidade do projeto. O *backend* gerencia a lógica de execução, as regras de negócio e o fluxo de dados do programa; o *frontend* exibe a interface do programa para o usuário e, por fim, o banco de dados relacional persiste as informações do programa, priorizando-se a consistência à disponibilidade dos dados.

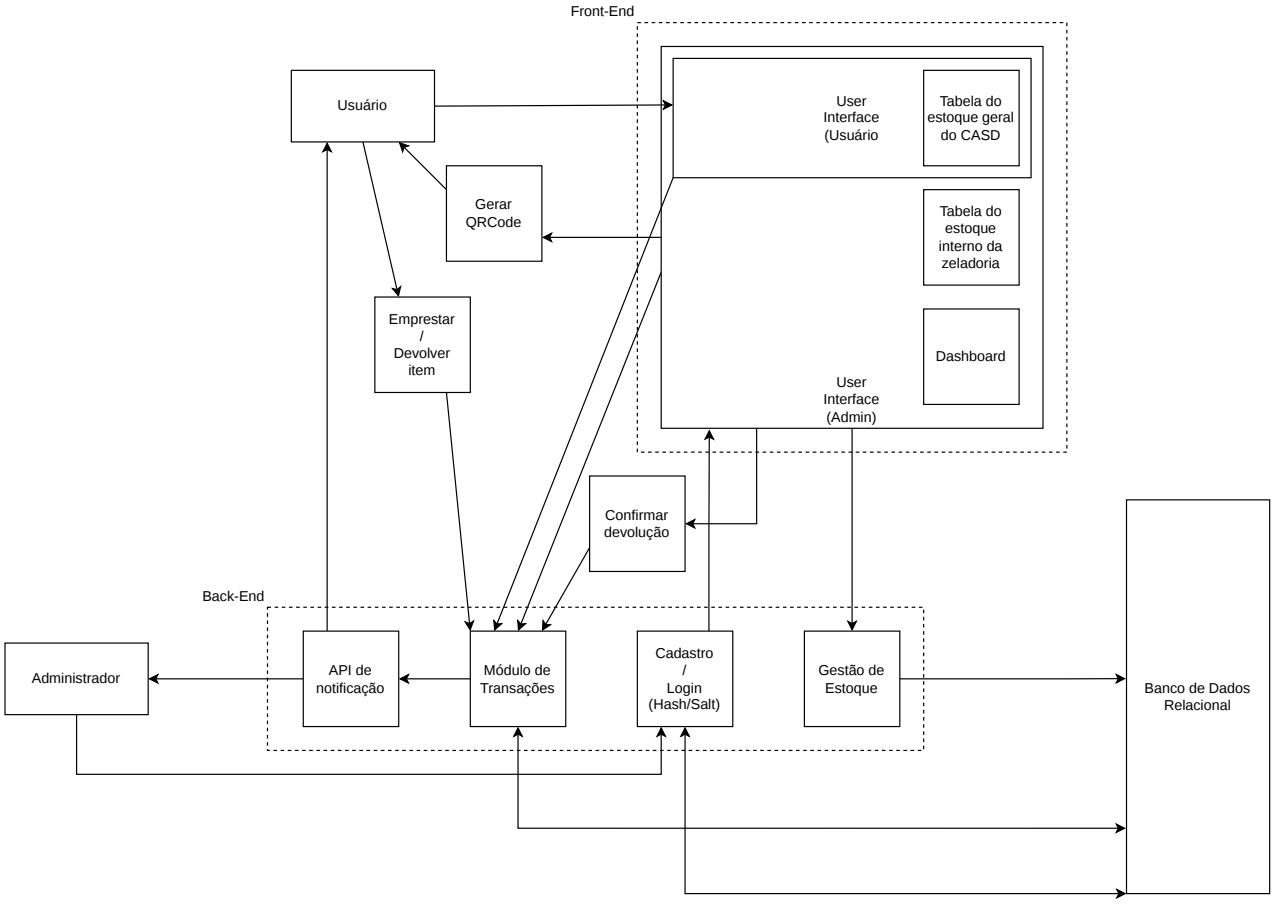


Figura 1: Diagrama de blocos da arquitetura do sistema.

1.4 Diagrama Entidade-Relacionamento (DER)

A Figura 2 exibe o DER do banco de dados do programa, considerando as informações que devem ser armazenadas conforme os requisitos, bem como os seus relacionamentos. Os itens serão armazenados em uma tabela única tabela e eles serão classificados entre itens gerais do CASD e

itens internos da zeladoria. As tabelas de uso e empréstimo armazenam o estado atual da utilização de um item interno da zeladoria ou do empréstimo de um item geral do CASD. As tabelas de *logs* fornecem os históricos dos usos e dos empréstimos para possibilitar a visualização e a verificação das transações após a sua conclusão. O cadastro de administradores é registrado na tabela `ADMIN`, incluindo a encriptação da senha. As informações do morador são armazenados em uma tabela, bem como o e-mail para envio de notificações.

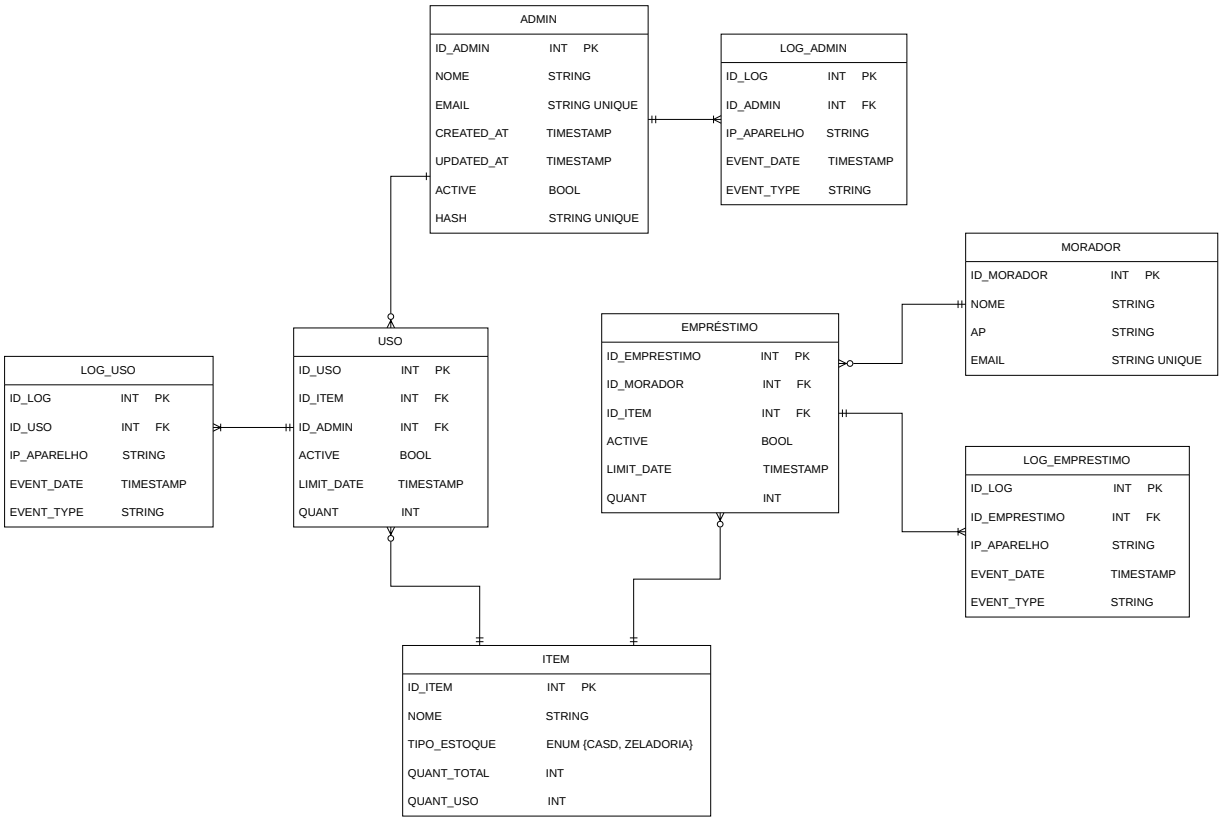


Figura 2: DER do banco de dados da aplicação.

1.5 Diagrama de Classes

A Figura 3 exibe o diagrama de classes proposto para a aplicação, considerando a descrição do programa, a arquitetura do sistema e o DER proposto. Tendo em vista a implementação do *backend* e *frontend* pelo *framework* Django, o diagrama de classes contempla as classes herdadas de `Model` (*Object-Relational Mapping*), que simboliza as instâncias de cada tabela do banco de dados descrito pela Figura 2, indicando os relacionamentos entre as classes a partir das chaves estrangeiras. Ainda, há as classes `Service`, que definem a lógica interna e as regras de negócio do sistema. Por fim, destaca-se que as entidades referentes à interface do usuário (*frontend*) não foram simbolizadas no diagrama de classes, pois, em Django, elas são implementadas a partir de funções ou por pequenas classes lidando com as requisições HTTP e renderizando HTMLs, o que tornaria o diagrama poluído e inconsistente com a implementação em código.

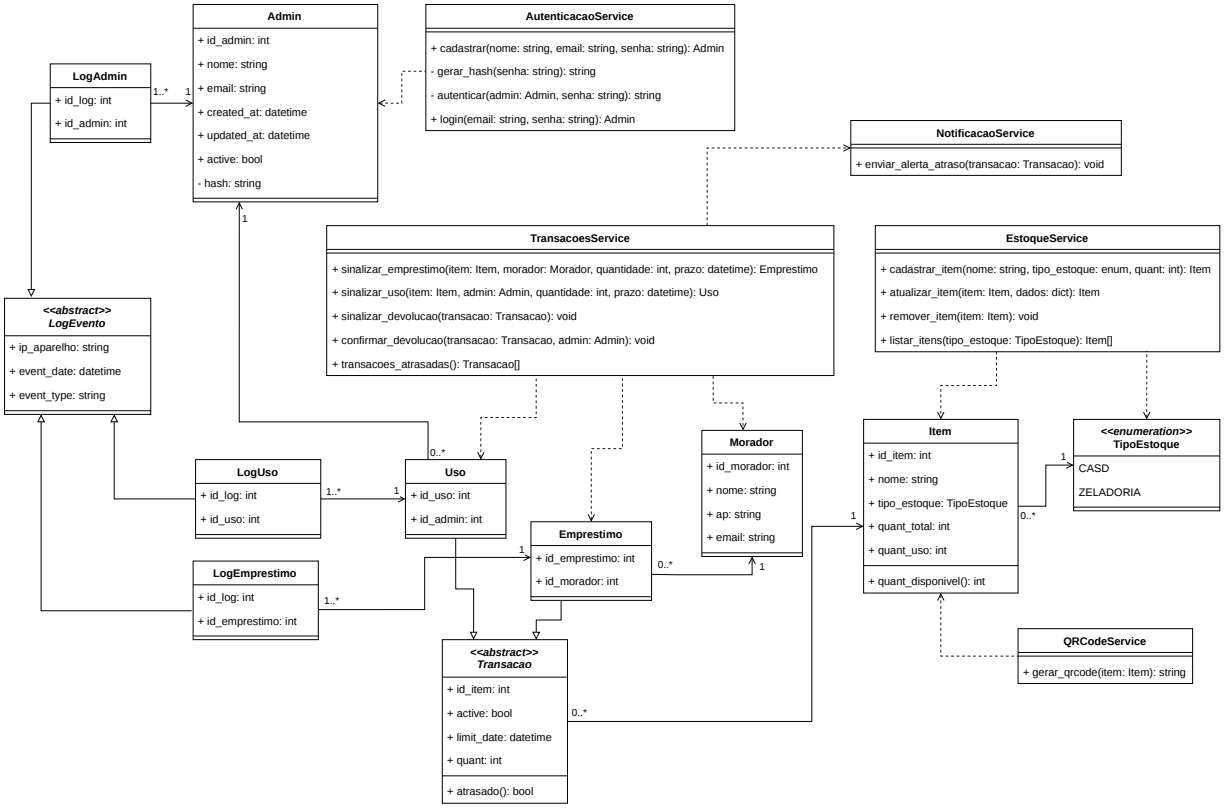


Figura 3: Diagrama de classes da aplicação.

2 O Projeto

2.1 Equipe de Desenvolvimento

A Tabela 1 exibe a equipe de desenvolvimento do projeto, descrevendo o papel, a experiência e a disponibilidade de cada membro na execução do projeto.

2.2 Cronograma

A Figura 4 exibe o cronograma proposto para o desenvolvimento do projeto. O cronograma é dividido em incrementos, considerando o *feedback* do cliente em cada uma das entregas. Adotou-se o prazo de nove semanas para elaboração dos quatro incrementos do produto, distribuindo as tarefas entre os desenvolvedores, o administrador de banco de dados e o gerente de projeto, bem como considerando o usuário da aplicação.

Tabela 1: Equipe de desenvolvimento do projeto.

Nome	Papel	Competências	Dedicação
Arthur Vianna	Desenvolvedor *backend*	Django, integração entre *site* e banco de dados, CRUD, autenticação, permissões de usuário, movimentação de itens, geração de *QR codes* e integração de API de notificação	4 horas semanais
Danilo Carlos	Desenvolvedor *frontend*	Django, desenvolvimento *frontend*, páginas de autenticação e de administrador, tabelas e *dashboards*, integração de CRUD e permissões, empréstimo, uso e devolução, *QR codes*	4 horas semanais
João Victor	Administrador de banco de dados	PostgreSQL, CRUD, *triggers*, autenticação, informações para *dashboard* e movimentação de itens	4 horas semanais
Matheus Reis	Gerente de projeto, DevOps	Django, gestão de requisitos, acompanhamento de entregas, comunicação com o cliente, gestão de *feedback*	4 horas semanais

Sigla	Tarefa	Dependência	Responsável	Semana 1	Semana 2	Semana 3	Semana 4	Semana 5	Semana 6	Semana 7	Semana 8	Semana 9
A.1	Implementar banco de dados em PostgreSQL	-	Admin Banco de Dados									
B.1	Criar funções de CRUD no banco de dados	A.1	Admin Banco de Dados									
C.1	Criar trigger para o banco de dados	B.1	Admin Banco de Dados									
D.1	Criar host local do site em Django	-	DevOps									
E.1	Criar verificador de atrasos	A.1	DevOps									
F.1	Implementar back-end que integre site e banco de dados em Django	A.1, D.1	Dev back-end									
-	Entregável: Documento de descrição do controle de configuração	-	Time									
G.1	Criar front-end de visualização de tabelas do site em Django	F.1	Dev front-end									
H.1	Teste das implementações	A.1 - G.1	DevOps									
X.1	Entrega e feedback do cliente - Entrega 1	H.1	Time + Usuário									
A.2	Correção de exigências do cliente	X.1	Time									
B.2	Implementar sistema de cadastro/login	A.1	Admin Banco de Dados									
C.2	Gerar informações de dashboard a partir do banco de dados	A.1	Admin Banco de Dados									
D.2	Criar front-end de página cadastro/login	D.1	Dev front-end									
E.2	Criar front-end de página de administrador	D.1	Dev front-end									
F.2	Criar em front-end página de dashboard para administrador	D.1	Dev front-end									
G.2	Implementar em back-end as funções CRUD	B.1, D.1	Dev back-end									
H.2	Implementar em back-end o candastro/login	B2	Dev back-end									
-	Entregável: Apresentação e entrega de plano de testes	-	Time									
I.2	Implementar em back-end permissões de usuário e de administrador	H.2	Dev back-end									
J.2	Integrar em front-end as permissões e funções CRUD	G.2, I.2	Dev front-end									
K.2	Teste das implementações	A.2 - J.2	DevOps									
X.2	Entrega e feedback do cliente - Entrega 2	K.2	Time + Usuário									
A.3	Correção de exigências do cliente	X.2	Time									
B.3	Implementar no banco de dados funções de movimentação de itens	A.1	Admin Banco de Dados									
C.3	Integrar em back-end as funções de movimentação	D.1, B.3	Dev back-end									
D.3	Integrar em front-end o empréstimo	C.3	Dev front-end									
E.3	Integrar em front-end o uso	C.3	Dev front-end									
F.3	Integrar gerador de QRCode em back-end	F.3	Dev back-end									
G.3	Integrar gerador de QRCode em front-end	F.3	Dev front-end									
H.3	Integrar no site o dashboard informativo	C.2, F.2	Dev front-end									
I.3	Teste das implementações	A.3 - I.3	DevOps									
X.3	Entrega e feedback do cliente - Entrega 3	I.3	Time + Usuário									
A.4	Correção de exigências do cliente	X.3	Time									
B.4	Integrar API de notificação no back-end	E.1	Dev back-end									
C.4	Implementar confirmação de retorno de item por administrador	C.3	Dev back-end									
D.4	Implementar confirmação de retorno no front-end	C.4	Dev front-end									
E.4	Teste das implementações	A.4 - D.4	DevOps									
X.4	Entrega ao cliente	E.4	Time + Usuário									
X.5	Correção de exigências do cliente	X.4	Time									
X.6	Entrega final e apresentação	X.5	Time									

Figura 4: Gráfico de Gantt do projeto, indicando as tarefas, as dependências e os responsáveis.